

DevOps Master Course - Volume 5: Kubernetes

DEVOPS MASTER COURSE

Volume 5

Kubernetes

From First Pod to Secure Production Platform Engineering

Course dimension	Definition
Audience	Beginners progressing to production-ready DevOps, SRE and platform engineering practice
Reference baseline	Kubernetes 1.36 concepts and stable APIs, with version-awareness for add-ons and managed services
Method	Understand -> declare -> apply -> observe -> break -> recover -> secure -> automate
Assessment	50 units, command labs, manifests, two capstones, interview preparation and final examination

Use a disposable local cluster for laboratories. Never apply unreviewed manifests or destructive commands to production.

How to Use This Volume

Kubernetes is learned by observing reconciliation. For every object, write the desired state, apply it, inspect the live state, change one condition, watch the controllers react, and document what happened. A resource being created does not prove the application is healthy.

Stage	Required action	Evidence
Learn	Read the mental model and define every unfamiliar field	Personal notes
Declare	Write or review YAML before applying it	Version-controlled manifest
Observe	Use get, describe, logs, events and metrics	Saved command output
Break	Introduce one controlled failure	Failure record
Recover	Restore service using a reversible method	Recovery steps
Validate	Answer questions and repeat the lab	80% or better

Laboratory Environment

- kubectl and one local cluster tool such as kind or minikube.
- Docker or another compatible container runtime used by the chosen local-cluster tool.
- Git for manifests, Helm charts, Kustomize overlays, runbooks and evidence.
- At least 8 GB RAM; 12 GB or more is preferable for multi-node and observability labs.
- Use dummy secrets only. Do not place real credentials in manifests, screenshots or repositories.

Current Version Note

This edition uses Kubernetes 1.36 as the current upstream reference series. Managed providers, distributions, CNI plugins, Ingress controllers, Gateway implementations, storage drivers and policy engines have independent support matrices. Always verify the target cluster version and implementation documentation before production use.

Course Map

Part	Focus	Units
I	Architecture, API model and local cluster operation	1-8
II	Pods, controllers and workload lifecycle	9-19
III	Services, traffic, networking and storage	20-31
IV	Configuration, scheduling, scaling and resilience	32-39
V	Security, packaging, extension and platform operations	40-46
VI	Capstones, interview mastery and final examination	47-50

Unit 1: Why Kubernetes Exists

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain container orchestration problems.
- Compare manual container operation with declarative orchestration.
- Identify when Kubernetes is and is not justified.

Why this matters

Running one container is simple. Operating many services across hosts requires scheduling, recovery, networking, configuration, rollout, scaling and policy.

The orchestration problem

- Select a healthy machine for each workload.
- Restart failed workloads and replace unhealthy nodes.
- Provide stable service discovery while Pods change.
- Roll out new versions without uncontrolled downtime.
- Attach persistent storage and enforce security policy.

Declarative control

Users submit desired state through the API. Controllers continuously compare desired and observed state and act to reduce the difference.

When not to use Kubernetes

A small, stable application may be simpler and cheaper on a managed platform, virtual machine or container service. Kubernetes adds operational complexity.

Command laboratory

Inspect kubectl

```
kubectl version --client
kubectl help
```

The client can be installed before a cluster exists.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

1. Write ten orchestration requirements for a three-service application.
2. List three situations where Kubernetes would be excessive.
3. Draw desired state -> API -> controller -> actual state.

Knowledge check

4. What is reconciliation?
5. Why are stable service identities needed?
6. What cost does Kubernetes add?

Completion evidence: _____

Unit 2: Cluster Architecture

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Identify control-plane and node components.
- Explain the purpose of etcd, API server, scheduler and controllers.
- Trace a Pod creation request.

Why this matters

Troubleshooting requires knowing which component owns each decision.

Control plane

- kube-apiserver exposes and validates the Kubernetes API.
- etcd stores cluster state and requires strong backup and access controls.
- kube-scheduler selects nodes for unscheduled Pods.
- kube-controller-manager runs core reconciliation controllers.
- cloud-controller-manager integrates supported cloud infrastructure where applicable.

Worker node

- kubelet reconciles Pods assigned to the node.
- The container runtime runs containers through the Container Runtime Interface.
- kube-proxy or an alternative dataplane implements Service traffic behavior.

Request flow

kubectl sends an authenticated API request. Admission runs. The object is stored. Controllers create dependent objects. The scheduler binds the Pod. The kubelet starts containers.

Command laboratory

Cluster inventory

```
kubectl cluster-info
kubectl get nodes -o wide
kubectl get --raw /readyz?verbose
```

The readyz endpoint may require sufficient permissions.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

7. Draw all core components.
8. Trace a Deployment to a running container.
9. Identify which component selects a node.

Knowledge check

10. Where is desired state stored?
11. Does the scheduler start containers?
12. What does kubelet reconcile?

Completion evidence: _____

Unit 3: Kubernetes API and Object Model

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Read apiVersion, kind, metadata, spec and status.
- Explain desired and observed state.
- Use object identity correctly.

Why this matters

Every Kubernetes action is an API operation on a resource.

Object structure

apiVersion and kind select a resource schema. metadata identifies and organizes it. spec expresses desired state. status reports observed state.

Names and UIDs

A name identifies an object within its scope. A UID distinguishes one lifecycle from a deleted and recreated object with the same name.

Generation and resourceVersion

generation tracks desired-state changes for many resources. resourceVersion supports concurrency and watch semantics; it is not a human version number.

Finalizers and owner references

Finalizers delay deletion until cleanup completes. Owner references support dependent garbage collection.

Reference manifest

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-model-demo
  namespace: default
  labels:
    app.kubernetes.io/name: volume5
data:
  message: "desired state lives in spec-like fields"
```

Command laboratory

Inspect object

```
kubectl apply -f configmap.yaml
kubectl get configmap api-model-demo -o yaml
kubectl explain configmap
```

Use kubectl explain to inspect the schema.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

13. Apply the object.

14. Record name, UID and resourceVersion.
15. Delete and recreate it, then compare UID.

Knowledge check

16. What is the difference between spec and status?
17. Why is UID important?
18. What does a finalizer do?

Completion evidence: _____

Unit 4: kubectl, kubeconfig and Context Safety

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Use contexts safely.
- Inspect kubeconfig.
- Apply namespace and output controls.

Why this matters

Many serious incidents occur because a valid command targets the wrong cluster or namespace.

Kubeconfig

Kubeconfig stores clusters, users, contexts and the current context. Authentication material may be embedded or referenced.

Context

A context combines cluster, user and optional default namespace.

Safety habits

Display current context, use explicit namespaces, use read-only commands first, prefer server-side dry run, and avoid sharing kubeconfig files.

Command laboratory

Inspect context

```
kubectl config current-context
kubectl config get-contexts
kubectl config view --minify
```

Review before changing resources.

Namespace-aware commands

```
kubectl get pods -A
kubectl get pods -n default
```

-A lists all namespaces where permitted.

Dry run

```
kubectl create deployment demo --image=nginx:1.27-alpine --dry-run=client -o yaml
```

Generate YAML without creating the object.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

19. Create a dedicated namespace in a local cluster.
20. Set a temporary context namespace.
21. Build a shell prompt or checklist that displays context.

Knowledge check

22. What does a context combine?
23. Why is an explicit namespace safer?
24. Is kubeconfig safe to publish?

Completion evidence: _____

Unit 5: Creating a Local Learning Cluster

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Create a disposable cluster.
- Validate nodes and system Pods.
- Delete and recreate the environment.

Why this matters

A reproducible local cluster allows safe failure simulation.

kind

kind runs Kubernetes nodes as containers and is excellent for CI and multi-node laboratories.

minikube

minikube provides a feature-rich local cluster with multiple drivers and add-ons.

Disposable design

Cluster configuration, manifests and setup scripts should be reproducible from version control.

Command laboratory

kind example

```
kind create cluster --name volume5
kubectl cluster-info --context kind-volume5
kubectl get nodes
```

Requires kind and a compatible container runtime.

Delete

```
kind delete cluster --name volume5
```

Deletion is expected in a disposable lab.

Alternative

```
minikube start
minikube status
```

Use either tool, not necessarily both.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.

- Record namespace, context and target cluster before every administrative change.

Hands-on lab

- Create a local cluster.
- Record Kubernetes server version and node details.
- Delete and recreate it from notes.

Knowledge check

- Why is kind useful in CI?
- What makes a cluster disposable?
- Which command confirms node readiness?

Completion evidence: _____

Unit 6: Namespaces, Labels and Annotations

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Organize resources with namespaces.
- Use labels for selection.
- Use annotations for non-identifying metadata.

Why this matters

Large clusters require clear ownership, environment boundaries and queryable metadata.

Namespaces

Namespaces scope names for many resources and provide a boundary for policy, quotas and access. They are not virtual clusters.

Labels

Labels are indexed key-value metadata used by selectors and controllers.

Annotations

Annotations store non-identifying metadata such as external links, checksums or tool configuration.

Recommended labels

app.kubernetes.io/name, instance, version, component, part-of and managed-by improve consistency.

Reference manifest

```
apiVersion: v1
kind: Namespace
metadata:
  name: volume5-lab
  labels:
    owner: sunny
    environment: learning
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: metadata-demo
  namespace: volume5-lab
  labels:
    app.kubernetes.io/name: metadata-demo
    app.kubernetes.io/component: configuration
  annotations:
    learning.devops.example/purpose: "label and annotation practice"
data:
```

Command laboratory

Query labels

```
kubectl apply -f metadata.yaml
kubectl get configmaps -n volume5-lab --show-labels
kubectl get configmaps -n volume5-lab -l app.kubernetes.io/name=metadata-demo
```

Selectors depend on labels, not annotations.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

31. Apply both resources.
32. Add and remove a label with `kubectl label`.
33. Query by two label conditions.

Knowledge check

34. Are namespaces hard security boundaries by themselves?
35. What uses label selectors?
36. When should annotations be used?

Completion evidence: _____

Unit 7: Declarative Management, Diff and Server-Side Apply

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Prefer declarative workflows.
- Preview changes.
- Understand field ownership.

Why this matters

Production changes must be reviewable, repeatable and attributable.

Imperative and declarative

Imperative commands are useful for learning and emergencies. Declarative files are better for review, automation and drift control.

Apply

`kubectl apply` sends desired configuration and merges fields according to apply semantics.

Server-side apply

The API server tracks field managers and detects conflicting ownership.

Diff

`kubectl diff` previews live-versus-proposed changes but still requires cluster access and may execute admission.

Command laboratory

Client validation

```
kubectl apply --dry-run=client -f app.yaml
```

Checks local syntax and known schema.

Server dry run

```
kubectl apply --server-side --dry-run=server -f app.yaml
```

Runs server validation and admission without persistence.

Preview and apply

```
kubectl diff -f app.yaml
```

```
kubectl apply --server-side --field-manager=volume5 -f app.yaml
```

Use a meaningful field manager.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

37. Create a ConfigMap declaratively.
38. Change one value and review diff.
39. Inspect managedFields.

Knowledge check

40. Why is server dry run stronger?
41. What is field ownership?
42. Why store manifests in Git?

Completion evidence: _____

Unit 8: API Discovery, kubectl Explain and Output Processing

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Discover resource types.
- Inspect schemas.
- Use structured output rather than fragile text parsing.

Why this matters

Reliable automation depends on API-aware queries.

Discovery

api-resources and api-versions show resources served by the current cluster.

Schema

kubectl explain follows OpenAPI schemas for built-in and installed resources.

Output

Use YAML, JSON, custom-columns, JSONPath or approved tools such as jq. Avoid parsing aligned human tables.

Command laboratory

Discovery

```
kubectl api-resources
kubectl api-versions | sort
kubectl explain deployment.spec.template.spec.containers
```

Results depend on installed APIs.

Structured output

```
kubectl get nodes -o custom-columns=NAME:.metadata.name,READY:.status.conditions[-1].status
kubectl get pods -A -o jsonpath='{range .items[*]}{.metadata.namespace}{"\t"}{.metadata.name}{"\n"}{end}'
```

JSONPath escaping varies by shell.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

43. Find the short name and scope of Deployments.
44. Explain container securityContext fields.
45. Export all namespace names as JSON.

Knowledge check

46. Why not parse default tables?
47. What does namespaced scope mean?
48. Where does kubectl explain get schema information?

Completion evidence: _____

Unit 9: Pods: The Smallest Deployable Unit

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Create and inspect Pods.
- Explain shared Pod context.
- Understand why controllers normally create Pods.

Why this matters

Every workload ultimately runs through Pods.

Pod model

A Pod contains one or more tightly coupled containers sharing a network namespace and optional volumes.

Ephemeral identity

Pods are replaceable. Their IP and UID can change. Stable access is provided by controllers and Services.

Direct Pods

Direct Pods are useful for learning, but production workloads normally use controllers.

Reference manifest

```
apiVersion: v1
```

```
kind: Pod
metadata:
  name: pod-demo
  namespace: volume5-lab
  labels:
    app: pod-demo
spec:
  containers:
  - name: web
    image: nginx:1.27-alpine
    ports:
    - name: http
      containerPort: 80
```

Command laboratory

Apply and inspect

```
kubectl apply -f pod.yaml
kubectl get pod pod-demo -n volume5-lab -o wide
kubectl describe pod pod-demo -n volume5-lab
```

Describe includes events.

Access

```
kubectl port-forward -n volume5-lab pod/pod-demo 8080:80
```

Use a second terminal for curl.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

49. Apply the Pod.
50. Record Pod IP, node and UID.
51. Delete it and observe that it is not recreated.

Knowledge check

52. What do containers in one Pod share?
53. Why is a Pod IP not a stable client endpoint?
54. Who recreates a direct Pod?

Completion evidence: _____

Unit 10: Multi-Container Pods and Init Containers

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Use init containers.
- Recognize sidecar and ambassador patterns.
- Avoid unrelated containers in one Pod.

Why this matters

Some components must share lifecycle, network and storage closely.

Init containers

Run sequentially to completion before application containers start.

Sidecars

Supporting containers may handle logging, proxying or synchronization. Use only when lifecycle and locality justify coupling.

Shared volumes

Containers in one Pod can exchange files through volumes.

Readiness coordination

Init containers can prepare dependencies, but applications still need runtime resilience.

Reference manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: init-demo
  namespace: volume5-lab
spec:
  volumes:
    - name: shared
      emptyDir: {}
  initContainers:
    - name: prepare
      image: busybox:1.36
      command: ["sh", "-c", "echo prepared > /work/status"]
      volumeMounts:
        - name: shared
          mountPath: /work
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "cat /work/status; sleep 3600"]
      volumeMounts:
        - name: shared
          mountPath: /work
```

Command laboratory

Observe sequence

```
kubectl apply -f init-demo.yaml
kubectl get pod init-demo -n volume5-lab -w
kubectl logs -n volume5-lab init-demo -c app
```

The app starts only after init success.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

55. Apply the Pod.
56. Break the init command and inspect status.
57. Fix it and verify app logs.

Knowledge check

58. Do init containers run in parallel?
 59. When should containers share a Pod?
 60. What volume type is used here?

Completion evidence: _____

Unit 11: Pod Lifecycle, Restart Policy and Termination

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Interpret Pod phases and container states.
- Understand restartPolicy.
- Design graceful termination.

Why this matters

Correct shutdown prevents request loss and data corruption.

Pod phase

Pending, Running, Succeeded, Failed and Unknown are broad summaries, not complete health states.

Container states

Waiting, Running and Terminated include reasons and exit codes.

Restart policy

Always, OnFailure or Never applies at Pod level, with controller behavior layered above it.

Termination

Kubelet runs preStop if configured, sends TERM, waits the grace period, then may send KILL.

Reference manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: lifecycle-demo
  namespace: volume5-lab
spec:
  terminationGracePeriodSeconds: 20
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "trap 'echo TERM received; exit 0' TERM; while true; do sleep 1; done"]
      lifecycle:
        preStop:
          exec:
            command: ["sh", "-c", "echo preStop started; sleep 3"]
```

Command laboratory

Observe termination

```
kubectl apply -f lifecycle.yaml
kubectl logs -f -n volume5-lab lifecycle-demo
```

Delete the Pod from another terminal and observe logs/events.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

61. Apply and delete gracefully.
62. Reduce grace period and compare.
63. Record exit reason and timing.

Knowledge check

64. What signal is normally sent first?
65. What is a Pod phase?
66. How does restartPolicy interact with a Deployment?

Completion evidence: _____

Unit 12: Probes: Startup, Readiness and Liveness

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Choose the correct probe.
- Avoid probe-induced outages.
- Debug unhealthy Pods.

Why this matters

Kubernetes needs distinct signals for startup, traffic readiness and deadlock recovery.

Startup probe

Protects slow-starting applications by delaying liveness and readiness evaluation until startup succeeds.

Readiness probe

Controls whether a Pod endpoint receives Service traffic. Failure does not normally restart the container.

Liveness probe

Triggers container restart after repeated failure. It must detect unrecoverable states, not temporary dependency outages.

Probe types

HTTP, TCP, gRPC and exec probes are available with version and implementation considerations.

Reference manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: probe-demo
  namespace: volume5-lab
spec:
  replicas: 2
  selector:
    matchLabels:
      app: probe-demo
  template:
    metadata:
      labels:
        app: probe-demo
    spec:
```

```
containers:
- name: web
  image: nginx:1.27-alpine
  ports:
  - containerPort: 80
  startupProbe:
    httpGet:
      path: /
      port: 80
    failureThreshold: 30
    periodSeconds: 2
  readinessProbe:
    httpGet:
      path: /
      port: 80
    periodSeconds: 5
  livenessProbe:
    httpGet:
      path: /
      port: 80
    periodSeconds: 10
```

Command laboratory

Inspect health

```
kubectl apply -f probes.yaml
kubectl get pods -n volume5-lab
kubectl describe pod -n volume5-lab -l app=probe-demo
```

Look for probe events and readiness conditions.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

67. Apply the Deployment.
68. Break readiness without breaking the process.
69. Compare endpoint membership and restart count.

Knowledge check

70. Which probe controls traffic?
71. Which probe restarts containers?
72. Why should liveness avoid dependency checks?

Completion evidence: _____

Unit 13: Requests, Limits and Quality of Service

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Set CPU and memory requests and limits.
- Explain scheduling versus runtime enforcement.
- Identify QoS classes.

Why this matters

Resource declarations protect nodes and make scheduling and scaling meaningful.

Requests

Scheduler uses requests to place Pods. They represent expected guaranteed capacity.

Limits

Runtime enforces memory limits and CPU throttling behavior. Memory excess can result in OOM termination.

QoS

Guaranteed, Burstable and BestEffort affect eviction priority under node pressure.

Right-sizing

Use observed workload data, load tests and safety margins. Limits that are too low cause instability; missing requests cause overcommit uncertainty.

Reference manifest

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: resource-demo
  namespace: volume5-lab
spec:
  replicas: 2
  selector:
    matchLabels:
      app: resource-demo
  template:
    metadata:
      labels:
        app: resource-demo
    spec:
      containers:
        - name: app
          image: nginx:1.27-alpine
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 500m
              memory: 128Mi

```

Command laboratory

Inspect resources

```

kubectl apply -f resources.yaml
kubectl get pod -n volume5-lab -l app=resource-demo -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.qosClass}{"\n"}{end}'

```

Metrics require a metrics pipeline.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

73. Apply the Deployment.
74. Create examples of all three QoS classes.
75. Inspect OOMKilled in a controlled memory test.

Knowledge check

76. Does a request cap usage?
77. What happens at a memory limit?
78. What defines Guaranteed QoS?

Completion evidence: _____

Unit 14: Deployments and ReplicaSets

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Create a Deployment.
- Explain ReplicaSet ownership.
- Scale declaratively.

Why this matters

Deployments manage stateless replicated applications and rollout history.

Deployment

Defines desired replicas and Pod template and manages ReplicaSets.

ReplicaSet

Maintains a replica count for a label selector. Normally managed by a Deployment.

Selector immutability

The selector must match template labels and should be designed carefully.

Scaling

Changing replicas adjusts desired state; autoscalers may also own that field.

Reference manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: volume5-lab
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27-alpine
          ports:
            - name: http
              containerPort: 80
```

Command laboratory

Apply and inspect

```
kubectl apply -f deployment.yaml
kubectl get deploy,rs,pods -n volume5-lab -l app=web
kubectl describe deployment web -n volume5-lab
```

Follow owner references.

Scale

```
kubectl scale deployment web -n volume5-lab --replicas=5
kubectl get pods -n volume5-lab -l app=web
```

Return the manifest to the intended declarative replica count afterward.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

79. Apply and scale.
80. Delete one Pod and observe replacement.
81. Trace Pod -> ReplicaSet -> Deployment ownership.

Knowledge check

82. Which controller creates Pods for a Deployment?
83. What happens when a Pod is deleted?
84. Why must selector match template labels?

Completion evidence: _____

Unit 15: Rolling Updates and Rollback

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Perform controlled rollouts.
- Inspect rollout status and history.
- Rollback safely.

Why this matters

Release safety depends on availability, validation and recovery.

RollingUpdate

maxUnavailable and maxSurge control replacement capacity.

Progress

progressDeadlineSeconds helps identify stalled rollouts.

Rollback

A Deployment can roll back to a prior ReplicaSet template. Data and external dependencies may still prevent full rollback.

Validation

Monitor readiness, error rate, latency, saturation and business signals.

Command laboratory

Update image

```
kubectl set image deployment/web web=nginx:1.27.1-alpine -n volume5-lab
kubectl rollout status deployment/web -n volume5-lab
```

Use an available image tag in the lab.

History and rollback

```
kubectl rollout history deployment/web -n volume5-lab
kubectl rollout undo deployment/web -n volume5-lab
```

Verify application behavior after rollback.

Pause

```
kubectl rollout pause deployment/web -n volume5-lab
kubectl rollout resume deployment/web -n volume5-lab
```

Useful for controlled multi-field changes.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

85. Perform a successful update.
86. Use an invalid image to create a stalled rollout.
87. Diagnose and roll back.

Knowledge check

88. What do maxSurge and maxUnavailable control?
89. Does rollback reverse database changes?
90. Which command shows rollout progress?

Completion evidence: _____

Unit 16: StatefulSets

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Explain stable identity and ordered management.
- Use headless Services.
- Understand persistent volume claims per replica.

Why this matters

Stateful applications often require stable names, storage and controlled lifecycle.

Identity

Pods receive stable ordinal names such as database-0.

Headless Service

clusterIP: None publishes Pod addresses without a virtual Service IP.

Volume claim templates

Each replica can receive an independent PVC that persists across Pod replacement.

Caution

StatefulSet does not make an application distributed, consistent or backed up.

Reference manifest

```

apiVersion: v1
kind: Service
metadata:
  name: stateful-demo
  namespace: volume5-lab
spec:
  clusterIP: None
  selector:
    app: stateful-demo
  ports:
    - name: peer
      port: 80
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: stateful-demo
  namespace: volume5-lab
spec:
  serviceName: stateful-demo
  replicas: 3
  selector:
    matchLabels:
      app: stateful-demo
  template:
    metadata:
      labels:
        app: stateful-demo
    spec:
      containers:
        - name: app
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80

```

Command laboratory

Observe identity

```

kubectl apply -f statefulset.yaml
kubectl get pods -n volume5-lab -l app=stateful-demo -w
kubectl get endpointslices -n volume5-lab -l kubernetes.io/service-name=stateful-demo

```

Observe ordered names.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.

- Record namespace, context and target cluster before every administrative change.

Hands-on lab

91. Apply the StatefulSet.
92. Delete ordinal 1 and observe recreation.
93. Resolve Pod DNS names from another Pod.

Knowledge check

94. What remains stable for StatefulSet Pods?
95. What does a headless Service do?
96. Does StatefulSet provide backups?

Completion evidence: _____

Unit 17: DaemonSets

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Run one Pod per eligible node.
- Control node selection.
- Operate node-level agents.

Why this matters

Logging, monitoring, storage and networking agents often need node-local presence.

DaemonSet behavior

Creates a Pod on every eligible node and adds Pods when new eligible nodes join.

Typical use

Log collectors, node metrics, CNI agents, storage plugins and security sensors.

Scheduling

Node selectors, affinity, tolerations and update strategy govern placement.

Risk

A broken DaemonSet can affect every node; use staged rollout and resource limits.

Reference manifest

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-reporter
  namespace: volume5-lab
spec:
  selector:
    matchLabels:
      app: node-reporter
  template:
    metadata:
      labels:
        app: node-reporter
    spec:
      containers:
        - name: reporter
          image: busybox:1.36
          command: ["sh", "-c", "echo running on $NODE_NAME; sleep 3600"]
          env:
            - name: NODE_NAME
```

```

valueFrom:
  fieldRef:
    fieldPath: spec.nodeName
resources:
  requests:
    cpu: 10m
    memory: 16Mi

```

Command laboratory

Inspect coverage

```

kubectl apply -f daemonset.yaml
kubectl get daemonset,pods -n volume5-lab -l app=node-reporter -o wide

```

Compare desired and available counts.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

97. Apply on a multi-node local cluster.
98. Restrict it to one labeled node.
99. Remove the label and observe reconciliation.

Knowledge check

100. How many Pods does a DaemonSet create?
101. What happens when a node joins?
102. Why are resource limits important?

Completion evidence: _____

Unit 18: Jobs and CronJobs

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Run finite workloads.
- Configure retries and deadlines.
- Schedule recurring work safely.

Why this matters

Backups, migrations, reports and batch processing require completion semantics rather than permanent replicas.

Job

Creates Pods until the required successful completions are reached.

Backoff and deadline

backoffLimit and activeDeadlineSeconds prevent uncontrolled retry or duration.

CronJob

Creates Jobs on a schedule. concurrencyPolicy controls overlapping runs.

Idempotency

Retries can execute work more than once. Batch tasks must tolerate duplication or use durable deduplication.

Reference manifest

```

apiVersion: batch/v1
kind: Job
metadata:
  name: checksum-job
  namespace: volume5-lab
spec:
  backoffLimit: 2
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: task
          image: busybox:1.36
          command: ["sh", "-c", "echo volume5 | sha256sum"]
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: minute-report
  namespace: volume5-lab
spec:
  schedule: "*/5 * * * *"
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: report
              image: busybox:1.36
              command: ["date"]

```

Command laboratory

Observe completion

```

kubectl apply -f jobs.yaml
kubectl get jobs,cronjobs -n volume5-lab
kubectl logs -n volume5-lab job/checksum-job

```

CronJob timing uses controller interpretation and configured time zone fields where supported.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

103. Run a successful and failed Job.
104. Set a deadline.
105. Manually create a Job from the CronJob.

Knowledge check

106. Why must batch work be idempotent?
 107. What does Forbid mean?
 108. Which restart policies are valid for Job Pods?

Completion evidence: _____

Unit 19: Workload Selection Guide

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Choose the correct controller.
- Recognize anti-patterns.
- Explain controller tradeoffs.

Why this matters

Using the wrong controller creates weak identity, unsafe updates or incorrect completion behavior.

Decision guide

- Deployment: stateless replicated service.
- StatefulSet: stable identity and storage per replica.
- DaemonSet: node-level agent.
- Job: finite batch work.
- CronJob: scheduled Job creation.
- Direct Pod: diagnostics or special static/control-plane cases, not normal application management.

Custom controllers

Operators and CRDs can encode domain-specific lifecycle beyond built-in controllers.

Command laboratory

Inventory workload types

```
kubectl get deployments,statefulsets,daemonsets,jobs,cronjobs -A
```

Review ownership and intended use.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

109. Classify ten application scenarios.
 110. Defend each controller choice.
 111. Convert a direct Pod into an appropriate controller.

Knowledge check

112. Which controller fits a node log agent?
 113. Which fits a database replica with stable disk?
 114. Why avoid direct Pods?

Completion evidence: _____

Unit 20: Services and Stable Access

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Create ClusterIP Services.
- Understand selectors and EndpointSlices.
- Debug missing endpoints.

Why this matters

Pods change identity; clients require a stable name and virtual endpoint.

Service

Defines stable virtual access to a set of endpoints.

Selector

A selector normally discovers matching Pods. Services without selectors can be paired with manually managed endpoints.

EndpointSlice

Stores scalable endpoint information used by networking implementations.

Ports

port is the Service port; targetPort identifies the backend port by number or name.

Reference manifest

```
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: volume5-lab
spec:
  selector:
    app: web
  ports:
    - name: http
      port: 80
      targetPort: http
  type: ClusterIP
```

Command laboratory

Apply and inspect

```
kubectl apply -f service.yaml
kubectl get service web -n volume5-lab
kubectl get endpointslices -n volume5-lab -l kubernetes.io/service-name=web
```

Endpoints should correspond to ready matching Pods.

Test

```
kubectl run curl --rm -it --restart=Never -n volume5-lab --image=curlimages/curl -- curl -sS http://web
```

Use a trusted diagnostic image.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.

- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

115. Apply the Service.
116. Break the selector and observe empty endpoints.
117. Repair and test DNS access.

Knowledge check

118. Why is a Service stable?
119. What does targetPort select?
120. What causes empty EndpointSlices?

Completion evidence: _____

Unit 21: Service Types and External Traffic

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Compare ClusterIP, NodePort, LoadBalancer and ExternalName.
- Understand externalTrafficPolicy.
- Avoid unnecessary exposure.

Why this matters

Service type determines how traffic enters or leaves cluster boundaries.

ClusterIP

Internal virtual IP and default type.

NodePort

Exposes a port on eligible nodes. Usually a building block rather than a complete production edge design.

LoadBalancer

Requests an external load balancer from a supporting implementation.

ExternalName

Returns a DNS CNAME-like mapping and does not proxy traffic.

Traffic policy

externalTrafficPolicy can preserve client source address and alter endpoint routing behavior.

Command laboratory

Change type safely

```
kubectl patch service web -n volume5-lab -p '{"spec":{"type":"NodePort"}}'
```

```
kubectl get service web -n volume5-lab
```

Use local lab only.

Inspect assigned port

```
kubectl get service web -n volume5-lab -o jsonpath='{.spec.ports[0].nodePort}{"\n"}'
```

Reachability depends on local cluster networking.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

121. Compare ClusterIP and NodePort.
122. Document LoadBalancer behavior in your environment.
123. Return the Service to ClusterIP.

Knowledge check

124. Which type is internal by default?
125. Does LoadBalancer work without an implementation?
126. What does ExternalName return?

Completion evidence: _____

Unit 22: DNS and Service Discovery

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Use cluster DNS names.
- Understand search domains.
- Troubleshoot resolution.

Why this matters

Most applications discover dependencies through DNS rather than Pod IPs.

Service DNS

A Service can resolve as `service.namespace.svc.cluster.local`, with search paths allowing shorter names.

Pod DNS

Pod DNS behavior depends on policy and cluster implementation.

Headless discovery

Headless Services return endpoint addresses directly.

Troubleshooting

Check Service, EndpointSlice, DNS Pod health, `resolv.conf`, query response and network policy.

Command laboratory

Inspect DNS

```
kubectrl run dns-test --rm -it --restart=Never -n volume5-lab --image=busybox:1.36 -- nslookup web
```

BusyBox DNS output is sufficient for a basic lab.

Resolver config

```
kubectrl run dns-shell --rm -it --restart=Never -n volume5-lab --image=busybox:1.36 -- cat /etc/resolv.conf
```

Observe search domains and nameserver.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

127. Resolve same-namespace short name.
128. Resolve cross-namespace FQDN.
129. Break Service name and document NXDOMAIN.

Knowledge check

130. What is the usual Service FQDN pattern?
131. What does a headless Service return?
132. Which resources should be checked when DNS resolves but traffic fails?

Completion evidence: _____

Unit 23: Ingress

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Understand Ingress resources and controllers.
- Route HTTP by host and path.
- Manage TLS references.

Why this matters

Ingress provides standardized HTTP and HTTPS routing, but requires a separate controller implementation.

Resource versus controller

Creating an Ingress object does nothing unless an Ingress controller watches it.

Rules

Host and path rules route traffic to Services.

IngressClass

Selects an implementation.

TLS

The Ingress references a Secret containing certificate and key; certificate issuance is commonly delegated to an add-on.

Reference manifest

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
  namespace: volume5-lab
spec:
  ingressClassName: nginx
  rules:
    - host: web.local.example
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
```

```
name: web
port:
  name: http
```

Command laboratory

Inspect controller support

```
kubectl get ingressclasses
kubectl apply --dry-run=server -f ingress.yaml
kubectl describe ingress web -n volume5-lab
```

Use the class installed in your lab.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

133. Install or enable a local Ingress controller.
134. Route one host to web.
135. Create a bad backend and inspect status/events.

Knowledge check

136. Does Kubernetes include a default Ingress controller?
137. What does pathType control?
138. Where is TLS material referenced?

Completion evidence: _____

Unit 24: Gateway API

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Explain GatewayClass, Gateway and Route roles.
- Compare Gateway API with Ingress.
- Recognize implementation dependency.

Why this matters

Gateway API provides role-oriented, extensible and protocol-aware traffic configuration.

Role separation

Infrastructure providers manage GatewayClasses, cluster operators create Gateways, and application teams attach Routes under policy.

Core resources

GatewayClass defines implementation, Gateway requests listeners and infrastructure, and HTTPRoute or other Route kinds define traffic rules.

Add-on status

Gateway API is installed separately and requires a conformant controller implementation.

Migration

Ingress remains widely used. Choose based on platform support and required capabilities.

Reference manifest

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: web
  namespace: volume5-lab
spec:
  parentRefs:
    - name: shared-gateway
      namespace: gateway-system
  hostnames:
    - web.local.example
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
        - name: web
          port: 80
```

Command laboratory

Discover APIs

```
kubectl api-resources | grep -i gateway
kubectl get gatewayclasses
```

The commands return resources only after CRDs and implementation are installed.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

139. Draw GatewayClass -> Gateway -> HTTPRoute.
140. Install a supported local implementation or perform server-side validation where available.
141. Inspect Route status conditions.

Knowledge check

142. Is Gateway API built into every cluster installation?
143. Who typically owns HTTPRoute?
144. What indicates whether a Route is accepted?

Completion evidence: _____

Unit 25: Kubernetes Network Model and CNI

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Explain Pod-to-Pod networking expectations.
- Understand CNI responsibility.
- Identify kube-proxy or replacement dataplane roles.

Why this matters

Networking behavior depends on both Kubernetes APIs and the installed implementation.

Network model

Pods receive addresses and should communicate without application-level NAT assumptions within the model.

CNI

A CNI plugin configures Pod interfaces, routes and often NetworkPolicy enforcement.

Service dataplane

kube-proxy or alternatives program forwarding for Service virtual addresses.

Implementation variance

Encapsulation, routing, eBPF, MTU and policy capabilities differ by CNI.

Command laboratory

Inspect addresses

```
kubectl get pods -A -o wide
kubectl get nodes -o wide
```

Compare Pod and node CIDRs where visible.

System components

```
kubectl get pods -n kube-system -o wide
```

Identify CNI and proxy components in the lab.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

145. Identify your CNI implementation.
146. Trace traffic from Pod to Service to backend.
147. Record MTU and route information where accessible.

Knowledge check

148. What does CNI configure?
149. Are all clusters implemented with the same dataplane?
150. What commonly implements Service forwarding?

Completion evidence: _____

Unit 26: NetworkPolicy

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Apply default-deny policy.
- Allow explicit flows.
- Understand enforcement prerequisites.

Why this matters

Namespace and Service boundaries do not automatically restrict Pod communication.

Policy model

NetworkPolicy selects Pods and defines allowed ingress and egress. Policies are additive.

Default deny

A policy selecting Pods with no allowed peers or ports isolates the relevant direction.

Implementation

The CNI must enforce NetworkPolicy. Unsupported policies may be accepted by the API but have no effect.

DNS egress

Egress policy must often allow DNS to the cluster resolver.

Reference manifest

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: volume5-lab
spec:
  podSelector: {}
  policyTypes: ["Ingress", "Egress"]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-client-to-web
  namespace: volume5-lab
spec:
  podSelector:
    matchLabels:
      app: web
  policyTypes: ["Ingress"]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              access: web
      ports:
        - protocol: TCP
          port: 80

```

Command laboratory

Apply and test

```

kubectl apply -f networkpolicy.yaml
kubectl get networkpolicy -n volume5-lab

```

Test from Pods with and without access=web.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

151. Confirm your CNI enforces policy.
152. Apply default deny.
153. Add DNS and application allowances incrementally.

Knowledge check

154. Are policies ordered?
155. Why can default-deny break DNS?
156. What happens if the CNI does not enforce policy?

Completion evidence: _____

Unit 27: Volumes and Ephemeral Storage

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Use emptyDir, projected and configuration volumes.
- Understand Pod-scoped storage.
- Control ephemeral-storage usage.

Why this matters

Pods often need shared scratch space and mounted configuration.

emptyDir

Created for the Pod and removed when the Pod is removed. It survives individual container restarts.

Projected volumes

Combine data from Secrets, ConfigMaps, downward API and service-account tokens.

Ephemeral storage

Writable layers, container logs and emptyDir on disk consume node ephemeral storage.

Memory-backed emptyDir

medium: Memory uses tmpfs and counts against memory resources.

Reference manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-demo
  namespace: volume5-lab
spec:
  volumes:
    - name: shared
      emptyDir:
        sizeLimit: 64Mi
  containers:
    - name: writer
      image: busybox:1.36
      command: ["sh", "-c", "while true; do date >> /shared/time; sleep 5; done"]
      volumeMounts:
        - name: shared
          mountPath: /shared
    - name: reader
      image: busybox:1.36
      command: ["sh", "-c", "tail -f /shared/time"]
      volumeMounts:
        - name: shared
```

Command laboratory

Observe shared data

```
kubectl apply -f volume-demo.yaml
kubectl logs -f -n volume5-lab volume-demo -c reader
```

Both containers share the Pod volume.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

157. Apply and verify sharing.
158. Restart one container if possible and verify data remains.
159. Delete the Pod and explain data loss.

Knowledge check

160. Does emptyDir survive Pod deletion?
161. What consumes node ephemeral storage?
162. How does memory-backed emptyDir affect memory?

Completion evidence: _____

Unit 28: PersistentVolumes, Claims and StorageClasses

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain PV and PVC binding.
- Use dynamic provisioning.
- Understand reclaim policy and access modes.

Why this matters

Persistent storage must outlive replaceable Pods and match infrastructure capabilities.

PVC

A namespaced request for capacity, access mode and optionally a StorageClass.

PV

A cluster-scoped storage resource bound to a claim.

StorageClass

Defines a provisioner and parameters for dynamic provisioning.

Reclaim policy

Delete or Retain controls what happens after claim release; behavior also depends on the storage system.

Access modes

RWO, ROX, RWX and RWOP describe attachment semantics, not application-level consistency.

Reference manifest

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data
  namespace: volume5-lab
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Command laboratory

Discover storage

```
kubectl get storageclass
kubectl get pv,pvc -A
```

A default StorageClass may dynamically provision the claim.

Apply claim

```
kubectl apply -f pvc.yaml
kubectl describe pvc app-data -n volume5-lab
```

Inspect events and selected class.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

163. Create a claim using the local default class.
164. Mount it in a Pod and persist data.
165. Replace the Pod and verify persistence.

Knowledge check

166. Which object is namespaced?
167. What selects the provisioner?
168. What does Retain mean operationally?

Completion evidence: _____

Unit 29: CSI, Snapshots and Storage Operations

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain CSI roles.
- Understand volume attachment and mount stages.
- Plan snapshots and restore.

Why this matters

Storage behavior is implemented by drivers with independent capabilities and failure modes.

CSI

Container Storage Interface separates Kubernetes from vendor storage implementations.

Components

Controller-side operations provision and attach; node-side operations stage and publish volumes.

Snapshots

VolumeSnapshot resources require installed snapshot CRDs, controller and driver support.

Operations

Backups require application consistency, retention, encryption and tested restore. A storage snapshot alone may not be application-consistent.

Command laboratory

Inspect CSI

```
kubectl get csidrivers
kubectl get volumeattachments
```

Permissions and local cluster capabilities vary.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

169. Identify the local storage driver.
170. Document supported access modes and expansion.
171. Design a database snapshot and restore test.

Knowledge check

172. Is CSI a storage product?
173. What is required for VolumeSnapshot?
174. Why can crash-consistent snapshots be insufficient?

Completion evidence: _____

Unit 30: Running Stateful Applications Safely

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Combine StatefulSet, Service and PVCs.
- Plan backups and failover.
- Respect application-specific consistency.

Why this matters

Kubernetes schedules processes; it does not automatically solve database replication, quorum or recovery.

Application responsibility

Replication, leader election, consistency and schema migration remain application or operator concerns.

Disruption

Quorum systems require topology, anti-affinity, PDBs and careful maintenance.

Backup

Back up data and metadata, encrypt copies, test restores and define recovery point and recovery time objectives.

Operators

Mature database operators can automate domain-specific lifecycle, but require evaluation and operational ownership.

Command laboratory

Inspect state

```
kubectl get statefulset,pods,pvc -n volume5-lab
kubectl describe statefulset stateful-demo -n volume5-lab
```

Correlate identity and storage.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

175. Design a three-replica stateful architecture.
176. Define backup and restore steps.
177. Simulate one Pod replacement, not data destruction.

Knowledge check

178. Does StatefulSet configure database replication?
179. What is RPO?
180. Why test restore?

Completion evidence: _____

Unit 31: Storage Troubleshooting

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Diagnose Pending PVCs and mount failures.
- Interpret events.
- Recover without data loss.

Why this matters

Storage failures often block scheduling or container startup and can threaten persistent data.

Pending claim

Check StorageClass, provisioner health, topology, requested size, access mode and quota.

Attach or mount failure

Check VolumeAttachment, node plugin, credentials, node limits, filesystem and stale attachment.

Permissions

Pod securityContext, fsGroup, container UID and storage behavior affect access.

Recovery

Stop unsafe writes, preserve evidence, snapshot or clone, repair on a copy and validate restore.

Command laboratory

PVC triage

```
kubectl describe pvc <claim> -n <namespace>
kubectl get events -n <namespace> --sort-by=.lastTimestamp
```

Replace placeholders.

Pod mount triage

```
kubectl describe pod <pod> -n <namespace>
kubectl get volumeattachments
```

Look for FailedMount and FailedAttachVolume.

Node plugin

```
kubectl get pods -A -o wide | grep -i csi
```

Identify controller and node components.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

181. Create a claim with an invalid StorageClass.
182. Diagnose Pending state.
183. Correct it without deleting valuable data.

Knowledge check

184. Where are provisioning errors reported?
185. What is a VolumeAttachment?
186. Why avoid deleting a bound volume during diagnosis?

Completion evidence: _____

Unit 32: ConfigMaps and Application Configuration

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Create ConfigMaps.
- Inject keys as environment or files.
- Understand update behavior.

Why this matters

Configuration should vary independently from immutable images.

Data model

ConfigMaps hold non-secret string or binary configuration.

Environment injection

Environment variables are captured when the container starts and do not update automatically.

Volume projection

Mounted ConfigMap data can update eventually, but applications must reload it; subPath mounts do not receive normal projected updates.

Rollout trigger

Teams often include a configuration checksum annotation in Pod templates.

Reference manifest

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: volume5-lab
data:
  APP_MODE: "learning"
  app.conf: |
    feature=true
    timeout=5
```

Command laboratory

Create and inspect

```
kubectl apply -f configmap.yaml
kubectl get configmap app-config -n volume5-lab -o yaml
```

Do not store credentials here.

Generate

```
kubectl create configmap generated-demo -n volume5-lab --from-literal=mode=test --dry-run=client -o yaml
```

Save generated YAML to Git before production use.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

187. Mount the ConfigMap as files.
188. Inject one value as environment.
189. Update the ConfigMap and compare behavior.

Knowledge check

190. Is ConfigMap encrypted secret storage?
191. Do environment values update live?
192. Why use a checksum rollout pattern?

Completion evidence: _____

Unit 33: Secrets and Workload Identity

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Use Secrets cautiously.
- Understand base64 encoding versus encryption.
- Prefer short-lived workload identity.

Why this matters

Credentials require stronger controls than ordinary configuration.

Secret object

Secret data is base64-encoded in API representation, not automatically encrypted by that encoding.

At-rest protection

Enable and manage API data encryption with key rotation where responsible for the control plane.

Access

Restrict RBAC, avoid broad list/watch permissions, prevent logs and environment dumps, and audit access.

Workload identity

Prefer provider or platform workload identity and short-lived tokens over static long-lived cloud keys.

External managers

External secret systems can synchronize or mount values; evaluate failure and rotation behavior.

Reference manifest

```
apiVersion: v1
kind: Secret
metadata:
  name: demo-secret
  namespace: volume5-lab
type: Opaque
stringData:
  username: demo
  password: not-a-real-password
```

Command laboratory

Apply dummy secret

```
kubectl apply -f secret.yaml
kubectl get secret demo-secret -n volume5-lab
```

Do not print real secret data.

Permissions check

```
kubectl auth can-i get secrets -n volume5-lab
kubectl auth can-i list secrets -n volume5-lab
```

List can expose many values to authorized clients.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

193. Use only dummy values.

194. Mount the Secret as a read-only file.
195. Write a rotation and exposure-response procedure.

Knowledge check

196. Is base64 encryption?
197. Why are list permissions sensitive?
198. What is workload identity?

Completion evidence: _____

Unit 34: Scheduling: Node Selection and Affinity

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Use nodeSelector and affinity.
- Understand hard and preferred rules.
- Avoid over-constraining scheduling.

Why this matters

Placement affects performance, cost, compliance and availability.

nodeSelector

Simple exact label matching.

Node affinity

requiredDuringSchedulingIgnoredDuringExecution is hard at scheduling; preferred rules influence scoring.

Pod affinity and anti-affinity

Place Pods near or away from other Pods based on labels and topology.

Risk

Too many hard constraints create Pending Pods and fragile capacity.

Reference manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: placement-demo
  namespace: volume5-lab
spec:
  replicas: 2
  selector:
    matchLabels:
      app: placement-demo
  template:
    metadata:
      labels:
        app: placement-demo
    spec:
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              podAffinityTerm:
                labelSelector:
                  matchLabels:
                    app: placement-demo
                topologyKey: kubernetes.io/hostname
```

```
containers:
  - name: app
    image: nginx:1.27-alpine
```

Command laboratory

Inspect placement

```
kubectl apply -f placement.yaml
kubectl get pods -n volume5-lab -l app=placement-demo -o wide
kubectl describe pod -n volume5-lab -l app=placement-demo
```

Single-node clusters cannot spread across hosts.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

199. Label a node for a lab role.
200. Use a preferred rule.
201. Create and diagnose an impossible required rule.

Knowledge check

202. What is a hard scheduling rule?
203. Does IgnoredDuringExecution evict existing Pods after label change?
204. Why prefer soft rules when possible?

Completion evidence: _____

Unit 35: Taints, Tolerations and Dedicated Nodes

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Explain taint effects.
- Add tolerations.
- Design dedicated workload pools.

Why this matters

Some nodes must repel general workloads or host special hardware and sensitive services.

Taints

A taint applies a key, value and effect to a node. NoSchedule blocks new placement, PreferNoSchedule is soft, and NoExecute can evict non-tolerating Pods.

Tolerations

Allow a Pod to be considered for a tainted node; they do not guarantee placement.

Dedicated pools

Combine taints with labels and node affinity to both repel other workloads and attract intended ones.

Command laboratory

Lab taint

```
kubectl taint nodes <node> dedicated=volume5:NoSchedule
kubectl describe node <node> | grep -A3 Taints
```

Replace the node and use a disposable cluster.

Remove

```
kubectl taint nodes <node> dedicated=volume5:NoSchedule-
```

The trailing hyphen removes the taint.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

- 205. Taint a worker node.
- 206. Observe a Pending Pod without toleration.
- 207. Add toleration and matching affinity.

Knowledge check

- 208. Does toleration attract a Pod?
 - 209. Which effect can evict existing Pods?
 - 210. Why combine taints and labels?
- Completion evidence: _____

Unit 36: Topology Spread and High Availability

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Spread replicas across failure domains.
- Use topologySpreadConstraints.
- Design capacity for disruptions.

Why this matters

Multiple replicas on one node or zone do not provide meaningful availability.

Failure domains

Hostname, zone and region labels represent infrastructure boundaries.

Topology spread

maxSkew, topologyKey and whenUnsatisfiable control distribution.

Capacity

Hard spread constraints need sufficient eligible nodes and spare resources.

Application design

Availability also requires readiness, graceful shutdown, dependency resilience and data consistency.

Reference manifest

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: spread-demo
  namespace: volume5-lab
spec:
  replicas: 3
  selector:
    matchLabels:
      app: spread-demo
  template:
    metadata:
      labels:
        app: spread-demo
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: spread-demo
      containers:
        - name: app
          image: nginx:1.27-alpine

```

Command laboratory

Inspect distribution

```

kubectl apply -f spread.yaml
kubectl get pods -n volume5-lab -l app=spread-demo -o wide

```

A multi-node cluster demonstrates spreading.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

211. Apply soft spread.
212. Change to hard DoNotSchedule and reduce eligible nodes.
213. Explain Pending behavior.

Knowledge check

214. What does maxSkew measure?
215. Why does one node limit availability?
216. What capacity does hard spreading require?

Completion evidence: _____

Unit 37: Autoscaling: HPA, VPA and Cluster Capacity

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain horizontal and vertical scaling.
- Use HPA prerequisites.
- Understand cluster autoscaling interaction.

Why this matters

Capacity should respond to demand while preserving stability and cost controls.

HPA

Changes replica count based on observed metrics and declared targets.

VPA

Recommends or adjusts resource requests through an add-on; update modes can restart Pods.

Cluster autoscaler

Adds or removes nodes based on unschedulable Pods and utilization constraints.

Prerequisites

Reliable requests, metrics, load behavior, stabilization and maximum capacity.

Caution

CPU-based scaling alone may not represent queues, latency or business demand.

Reference manifest

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web
  namespace: volume5-lab
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 10
  behavior:
    scaledDown:
      stabilizationWindowSeconds: 300
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

Command laboratory

Metrics readiness

```
kubectl top nodes
kubectl top pods -n volume5-lab
```

Requires metrics-server or another metrics API provider.

Apply HPA

```
kubectl apply -f hpa.yaml
kubectl get hpa -n volume5-lab -w
```

The target Deployment needs CPU requests.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

217. Install or enable metrics-server in a local lab.
218. Generate controlled load.
219. Observe scale up and stabilization.

Knowledge check

220. Why does CPU utilization require requests?
221. What scales nodes?
222. What risk comes from rapid scale down?

Completion evidence: _____

Unit 38: Disruptions, Eviction and PodDisruptionBudgets

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Distinguish voluntary and involuntary disruption.
- Use PDBs correctly.
- Drain nodes safely.

Why this matters

Maintenance should not remove too much application capacity at once.

Disruption types

Voluntary disruptions include drain and rollout; involuntary disruptions include node failure.

PDB

Limits simultaneous voluntary disruption for selected Pods through minAvailable or maxUnavailable.

Limits

PDB does not create replicas, prevent node failure or guarantee application health.

Drain

Evicts workload Pods while respecting PDB and controller ownership, subject to flags and special Pods.

Reference manifest

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web
  namespace: volume5-lab
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: web
```

Command laboratory

Inspect budget

```
kubectl apply -f pdb.yaml
kubectl get pdb web -n volume5-lab
kubectl describe pdb web -n volume5-lab
```

Healthy replicas determine allowed disruptions.

Drain lab

```
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
kubectl uncordon <node>
```

Use only in a disposable multi-node cluster and review every flag.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

- 223. Apply the PDB.
- 224. Reduce replicas and inspect disruptionsAllowed.
- 225. Drain one lab worker and recover.

Knowledge check

- 226. Does a PDB protect against sudden node failure?
- 227. What does cordon do?
- 228. Why can drain block?

Completion evidence: _____

Unit 39: ResourceQuota and LimitRange

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Control namespace consumption.
- Set default requests and limits.
- Prevent unbounded object growth.

Why this matters

Multi-tenant clusters need resource governance and predictable capacity.

ResourceQuota

Limits aggregate namespace resources such as requests, limits, object counts and storage.

LimitRange

Sets defaults and min/max constraints for individual containers, Pods or claims.

Admission effect

Objects that violate policy are rejected before persistence.

Operations

Quotas require capacity planning, exception process and monitoring.

Reference manifest

```

apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  namespace: volume5-lab
spec:
  limits:
    - type: Container
      defaultRequest:
        cpu: 100m
        memory: 64Mi
      default:
        cpu: 500m
        memory: 256Mi
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute
  namespace: volume5-lab
spec:
  hard:
    requests.cpu: "2"
    requests.memory: 2Gi
    limits.cpu: "4"
    limits.memory: 4Gi
    pods: "20"

```

Command laboratory

Apply and inspect

```

kubectl apply -f quota.yaml
kubectl describe limitrange defaults -n volume5-lab
kubectl describe resourcequota compute -n volume5-lab

```

Existing Pods are not retroactively changed.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

229. Apply defaults.
230. Create a Pod without resources and inspect injected values.
231. Attempt to exceed quota.

Knowledge check

232. Which object sets defaults?
233. Which limits aggregate namespace use?
234. Are existing objects retroactively modified?

Completion evidence: _____

Unit 40: ServiceAccounts, Authentication and RBAC

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Use ServiceAccounts.
- Write least-privilege Roles.
- Test authorization.

Why this matters

Every human and workload should receive only the permissions required.

Authentication

Clusters may use certificates, OIDC, cloud identity or other methods. Authentication identifies the caller.

Authorization

RBAC evaluates allowed verbs on API resources.

ServiceAccount

Provides workload identity inside a namespace. Modern projected tokens are short-lived and audience-bound when configured.

RBAC objects

Role and RoleBinding are namespaced. ClusterRole and ClusterRoleBinding can grant cluster-wide or reusable permissions.

Least privilege

Avoid wildcard verbs/resources and unnecessary secrets access.

Reference manifest

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: config-reader
  namespace: volume5-lab
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: config-reader
  namespace: volume5-lab
rules:
- apiGroups: [""]
  resources: ["configmaps"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: config-reader
  namespace: volume5-lab
subjects:
- kind: ServiceAccount
  name: config-reader
  namespace: volume5-lab
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: config-reader
```

Command laboratory

Apply and test

```
kubectl apply -f rbac.yaml
kubectl auth can-i list configmaps -n volume5-lab --as=system:serviceaccount:volume5-lab:config-reader
kubectl auth can-i delete pods -n volume5-lab --as=system:serviceaccount:volume5-lab:config-reader
```

Impersonation requires permission for the administrator running the command.

Review bindings

```
kubectl get rolebindings,clusterrolebindings -A
```

Cluster-wide output may require elevated permission.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

- 235. Grant read-only ConfigMap access.
- 236. Prove Pod deletion is denied.
- 237. Remove one verb and retest.

Knowledge check

- 238. What is the difference between authentication and authorization?
- 239. Which RBAC objects are namespaced?
- 240. Why avoid wildcard permissions?

Completion evidence: _____

Unit 41: Security Contexts and Pod Hardening

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Run containers as non-root.
- Drop capabilities.
- Use read-only filesystems and seccomp.

Why this matters

Workload configuration is a primary security boundary.

Pod and container context

Security settings include UID/GID, fsGroup, privilege, capabilities, SELinux and seccomp.

Non-root

Set runAsNonRoot and a known UID compatible with the image.

Capabilities

Drop ALL and add only required capabilities.

Filesystem

Use readOnlyRootFilesystem with explicit writable volumes.

Seccomp

RuntimeDefault provides a baseline system-call profile on supporting runtimes.

Reference manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: hardened
  namespace: volume5-lab
spec:
  securityContext:
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: app
      image: nginxinc/nginx-unprivileged:1.27-alpine
      ports:
        - containerPort: 8080
      securityContext:
        allowPrivilegeEscalation: false
        runAsNonRoot: true
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]
      volumeMounts:
        - name: tmp
          mountPath: /tmp
        - name: cache
          mountPath: /var/cache/nginx
  volumes:
    - name: tmp
      emptyDir: {}
    - name: cache
      emptyDir: {}
```

Command laboratory

Validate

```
kubectl apply --dry-run=server -f hardened.yaml
kubectl apply -f hardened.yaml
kubectl get pod hardened -n volume5-lab
```

Image paths and runtime requirements must be tested.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

241. Run a non-root Pod.
242. Attempt a privileged operation.
243. Document required writable paths.

Knowledge check

244. What does allowPrivilegeEscalation control?
245. Why drop ALL capabilities?

246. What does RuntimeDefault select?

Completion evidence: _____

Unit 42: Pod Security Standards and Admission

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain Privileged, Baseline and Restricted levels.
- Label namespaces for enforcement.
- Plan safe rollout.

Why this matters

Pod Security Admission provides built-in checks against standardized security profiles.

Policy levels

Privileged is unrestricted, Baseline blocks known privilege escalations, and Restricted applies stronger hardening requirements.

Modes

enforce rejects violations, audit records them, and warn informs clients.

Version labels

Policies can be pinned to a Kubernetes version or use latest. Production upgrades should test policy effects.

Rollout

Start with audit and warn, inventory violations, remediate, then enforce with controlled exceptions.

Command laboratory

Label namespace

```
kubectl label namespace volume5-lab pod-security.kubernetes.io/warn=restricted
pod-security.kubernetes.io/audit=restricted --overwrite
```

Observe warnings before enforcing.

Enforce baseline

```
kubectl label namespace volume5-lab pod-security.kubernetes.io/enforce=baseline --overwrite
```

Test workloads before applying to shared namespaces.

Inspect labels

```
kubectl get namespace volume5-lab --show-labels
```

Record policy version strategy.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

247. Enable restricted warn and audit.

248. Apply a violating Pod and capture warnings.

249. Harden the Pod, then plan enforcement.

Knowledge check

- 250. What are the three policy levels?
- 251. What does warn do?
- 252. Why pin policy versions during upgrades?

Completion evidence: _____

Unit 43: Admission Control and Policy Engines

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Understand mutating and validating admission.
- Use policies responsibly.
- Avoid hidden operational coupling.

Why this matters

Admission is the enforcement point between authenticated API request and persistence.

Admission chain

After authentication and authorization, mutating admission can change an object and validating admission can reject it.

Built-in controls

Resource quotas, Pod Security Admission and other controllers enforce cluster rules.

Webhooks

External webhooks add policy but can affect API availability and latency. They need timeouts, failure-policy decisions and highly available endpoints.

Policy as code

ValidatingAdmissionPolicy and ecosystem engines can express reusable rules depending on cluster version and requirements.

Command laboratory

Discover webhooks

```
kubectl get mutatingwebhookconfigurations,validatingwebhookconfigurations
```

Review ownership and failurePolicy.

Server dry run

```
kubectl apply --dry-run=server -f app.yaml
```

This exercises admission without storing the object.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

- 253. Inventory admission webhooks in a lab.
- 254. Write a policy requirement such as mandatory owner label.
- 255. Define failure behavior and emergency bypass governance.

Knowledge check

256. When does admission occur?
 257. What risk does failurePolicy Fail introduce?
 258. Why use server dry run?

Completion evidence: _____

Unit 44: Helm Package Management

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Install and inspect charts.
- Template before release.
- Manage values and upgrades.

Why this matters

Helm packages related resources and supports reusable configuration and release history.

Chart

Contains templates, default values and metadata.

Release

An installed chart instance in a namespace.

Values

Values customize templates. Keep environment values reviewed and avoid storing secrets in plain files.

Lifecycle

helm upgrade --install is common, but hooks, CRDs and stateful changes require care.

Validation

Render templates, lint, inspect diff through approved tooling and test rollback.

Command laboratory

Create chart

```
helm create volume5-chart
helm lint volume5-chart
helm template demo volume5-chart --namespace volume5-lab > rendered.yaml
```

Review rendered resources.

Install

```
helm upgrade --install demo ./volume5-chart -n volume5-lab --create-namespace
helm list -n volume5-lab
helm history demo -n volume5-lab
```

Use a lab cluster.

Rollback

```
helm rollback demo 1 -n volume5-lab
```

Rollback behavior depends on chart and data changes.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

- 259. Create a chart for a small Deployment and Service.
- 260. Override image and replicas.
- 261. Upgrade, inspect history and roll back.

Knowledge check

- 262. What is a release?
- 263. Why render templates first?
- 264. Does Helm rollback reverse external data changes?

Completion evidence: _____

Unit 45: Kustomize and Environment Overlays

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Build bases and overlays.
- Apply patches without templating.
- Avoid configuration duplication.

Why this matters

Kustomize supports declarative environment customization using native YAML transformations.

Base

Reusable resource set.

Overlay

References a base and applies patches, name changes, labels or image substitutions.

Built into kubectl

`kubectl apply -k` and `kubectl kustomize` process kustomization directories.

Governance

Keep overlays small, review generated output and avoid deeply layered complexity.

Command laboratory

Structure

```
mkdir -p kustomize/base kustomize/overlays/dev
```

Create resource and kustomization files.

Render

```
kubectl kustomize kustomize/overlays/dev
kubectl apply -k kustomize/overlays/dev
```

Review output before apply.

Image substitution

```
kustomize edit set image app=registry.example/app:1.2.3
```

Run within the intended kustomization directory if standalone kustomize is installed.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

- 265. Create a base Deployment and Service.
- 266. Create dev and production overlays.
- 267. Compare rendered YAML.

Knowledge check

- 268. Does Kustomize require template syntax?
- 269. What belongs in a base?
- 270. Why review generated output?

Completion evidence: _____

Unit 46: CRDs, Operators, Observability, GitOps and Cluster Operations

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Understand extension APIs.
- Explain operator reconciliation.
- Design observable and upgradeable platforms.
- Use GitOps safely.

Why this matters

Production Kubernetes extends core APIs and requires lifecycle ownership beyond application manifests.

CRDs and operators

A CustomResourceDefinition adds an API type. A controller or operator implements behavior. Installing a CRD alone does not create reconciliation.

Observability

Collect control-plane and node metrics, workload metrics, events, audit logs, application logs and traces. Define SLOs and actionable alerts.

GitOps

A controller reconciles cluster state from a Git source. Changes should flow through reviewed commits, with controlled secrets and emergency procedures.

etcd and control plane

Self-managed clusters require encrypted etcd backups, restore tests, certificate management and highly available control-plane design.

Upgrades

Respect supported version skew, API deprecations, add-on compatibility and sequential minor upgrade policy.

Managed Kubernetes

Cloud providers manage selected control-plane responsibilities, but customers still own workloads, access, networking choices, policy, cost and data protection.

Command laboratory

CRD discovery

```
kubectl get crds
kubectl api-resources --api-group=<group>
```

Replace the group.

Events and metrics

```
kubectl get events -A --sort-by=.lastTimestamp
kubectl top nodes
kubectl top pods -A
```

Metrics require a provider.

API deprecation check

```
kubectl get --raw /version
kubectl api-resources
```

Use provider and upstream upgrade guides for exact target versions.

GitOps inventory

```
kubectl get deployments -A | grep -Ei 'argo|flux'
```

Names vary by implementation.

How companies use this

Professional teams keep manifests and policies in version control, validate them in CI, apply least privilege, observe reconciliation and maintain tested rollback and incident procedures.

Common mistakes and prevention

- Do not test destructive commands on a production cluster.
- Do not treat object creation as proof of application readiness.
- Record namespace, context and target cluster before every administrative change.

Hands-on lab

271. Install one harmless sample CRD and inspect discovery.
272. Design a dashboard for API server, nodes and workloads.
273. Write an etcd backup and restore test plan for a self-managed cluster.
274. Create a GitOps repository structure with environments and promotion flow.
275. Write an upgrade checklist covering control plane, nodes, CNI, CSI, Ingress/Gateway, metrics and policy.

Knowledge check

276. What is missing when only a CRD is installed?
277. Who owns workload security in managed Kubernetes?
278. Why are sequential minor upgrades important?
279. What is the difference between monitoring and an SLO?
280. What happens when Git and live state differ under GitOps?

Completion evidence: _____

Unit 47: Capstone 1: Production Application Platform

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Deploy a secure multi-service application.
- Implement traffic, configuration, scaling and resilience.
- Produce operational evidence.

Why this matters

This capstone integrates the application-facing Kubernetes primitives.

Scenario

Deploy a frontend, API and stateful dependency to a local or managed learning cluster.

Mandatory deliverables

- Namespaces and recommended labels.
- Deployments with probes, resources and security contexts.
- StatefulSet or external managed data service with tested backup.
- Services and Ingress or Gateway routing.
- ConfigMaps and secure secret delivery.
- NetworkPolicy default deny with explicit flows.
- HPA and PDB where supported.
- Helm chart or Kustomize overlays.
- Dashboards, alerts, runbook and rollback procedure.

Command laboratory

Evidence inventory

```
kubectl get all,configmaps,secrets,pvc,networkpolicies,pdb,hpa -n <namespace>
kubectl get events -n <namespace> --sort-by=.lastTimestamp
```

Redact secret data from evidence.

Hands-on lab

281. Design the architecture and traffic matrix.
282. Build and deploy all manifests through Git.
283. Run load and failure tests.
284. Perform a rolling update and rollback.
285. Back up and restore persistent state.
286. Produce a final operations handbook.

Knowledge check

287. Can every Pod run non-root?
288. Are only required network flows allowed?
289. Can the application survive one Pod disruption?

Completion evidence: _____

Unit 48: Capstone 2: Secure Multi-Tenant Cluster Baseline

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Design a governed cluster platform.
- Enforce tenancy, policy and observability.
- Create onboarding and incident workflows.

Why this matters

Platform engineering provides safe, repeatable paths for many application teams.

Scenario

Create a baseline for development, staging and production teams on a shared or simulated cluster.

Mandatory deliverables

- Namespace onboarding template.
- RBAC roles and separation of duties.
- ResourceQuota and LimitRange.

- Pod Security Admission rollout.
- Default-deny NetworkPolicy.
- Approved Ingress or Gateway pattern.
- StorageClass and backup policy.
- Admission and image policy design.
- Audit, metrics, logging and alerting plan.
- Upgrade, disaster recovery and break-glass runbooks.

Command laboratory

Baseline audit

```
kubectl get namespaces --show-labels
kubectl get resourcequota,limitrange -A
kubectl get networkpolicy -A
kubectl get rolebindings,clusterrolebindings -A
```

Review only with authorized permissions.

Hands-on lab

290. Create two tenant namespaces.
291. Prove one tenant cannot read the other's ConfigMaps.
292. Enforce restricted policy in one namespace.
293. Demonstrate quota rejection and permitted deployment.
294. Run an access review and produce evidence.
295. Simulate an emergency access event and post-incident review.

Knowledge check

296. What prevents cross-tenant API access?
297. What prevents cross-tenant network access?
298. How is break-glass access governed?

Completion evidence: _____

Unit 49: Kubernetes Interview Mastery

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Answer foundational and senior questions.
- Troubleshoot whiteboard scenarios.
- Explain tradeoffs and ownership.

Why this matters

Strong interviews test mental models, evidence-based diagnosis and safe production judgment.

Core question bank

- What happens after kubectl applies a Deployment?
- Pod versus Deployment versus StatefulSet.
- Service, Ingress and Gateway API.
- Requests, limits, QoS and eviction.
- Readiness versus liveness versus startup probes.
- How DNS and NetworkPolicy interact.
- PV, PVC, StorageClass and CSI.
- RBAC versus Pod Security versus NetworkPolicy.
- How to diagnose a Pending Pod.
- How to diagnose CrashLoopBackOff.
- How to perform a safe cluster upgrade.
- How GitOps changes incident response.
- How to secure a multi-tenant cluster.
- How to design a highly available application.

Answer framework

Clarify scope and implementation, identify object ownership, inspect desired and observed state, collect events and logs, propose reversible mitigation, validate, then discuss long-term prevention.

Command laboratory

Scenario triage

```
kubectl get pod <pod> -n <namespace> -o wide
kubectl describe pod <pod> -n <namespace>
kubectl logs <pod> -n <namespace> --all-containers --previous
kubectl get events -n <namespace> --sort-by=.lastTimestamp
```

Choose commands based on actual state.

Hands-on lab

299. Write answers to all questions.
300. Practice three architecture diagrams.
301. Diagnose five intentionally broken manifests.
302. Record a mock interview and review clarity.

Knowledge check

303. Do your answers distinguish core API from implementation?
304. Do they include validation and rollback?
305. Can you explain reconciliation without jargon?

Completion evidence: _____

Unit 50: Final Practical Examination and Completion Review

Completion: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Demonstrate integrated competence.
- Create a reproducible portfolio.
- Identify and close remaining gaps.

Why this matters

Completion means you can design, deploy, secure, observe and troubleshoot Kubernetes systems independently.

Practical examination

- Create or connect to a disposable cluster safely.
- Deploy a secure application with Deployment, Service, probes and resources.
- Add configuration and dummy secret delivery.
- Create persistent storage and verify replacement.
- Apply network isolation and RBAC.
- Package with Helm or Kustomize.
- Perform rollout, rollback, scale and controlled disruption.
- Diagnose Pending, ImagePullBackOff, CrashLoopBackOff and Service routing failures.
- Write an upgrade, backup and incident runbook.

Passing standard

Complete all mandatory tasks, score at least 80% on theory review, and provide evidence another engineer can reproduce.

Command laboratory

Final evidence

```
kubectl version
kubectl cluster-info
```

```
kubectl get nodes -o wide  
kubectl get all -A  
kubectl get events -A --sort-by=.lastTimestamp
```

Redact sensitive data before sharing evidence.

Hands-on lab

- 306. Complete the examination in a disposable environment.
- 307. Create a portfolio folder with manifests, diagrams, outputs and lessons learned.
- 308. Repeat weak units and re-test.

Knowledge check

- 309. Can you explain every object owner?
- 310. Can you restore state after replacement?
- 311. Can another engineer follow your runbook?

Completion evidence: _____

Kubernetes Troubleshooting Quick Reference

Symptom	First evidence	Likely domains
Pending Pod	describe Pod, events, scheduler messages	resources, constraints, PVC, taints, quota
ImagePullBackOff	describe Pod, image reference, pull secret	registry, authentication, DNS, tag or digest
CrashLoopBackOff	logs --previous, exit code, command, probes	application, config, secret, permissions, OOM
Not Ready	probe events, endpoints, local health request	readiness path, listener, dependency
Service unreachable	Service, EndpointSlice, DNS, NetworkPolicy	selector, targetPort, readiness, dataplane
PVC Pending	describe PVC, StorageClass, CSI Pods	provisioner, class, topology, quota
Node NotReady	node conditions, kubelet/runtime/CNI logs	node, runtime, network, disk, certificates
Rollout stalled	rollout status, ReplicaSets, Pod events	image, probes, capacity, policy

Master Completion Tracker

Unit	Title	Status	Completion date
1	Why Kubernetes Exists	Not started	
2	Cluster Architecture	Not started	
3	Kubernetes API and Object Model	Not started	
4	kubectl, kubeconfig and Context Safety	Not started	
5	Creating a Local Learning Cluster	Not started	
6	Namespaces, Labels and Annotations	Not started	
7	Declarative Management, Diff and Server-Side Apply	Not started	
8	API Discovery, kubectl Explain and Output Processing	Not started	
9	Pods: The Smallest Deployable Unit	Not started	
10	Multi-Container Pods and Init Containers	Not started	
11	Pod Lifecycle, Restart Policy and Termination	Not started	
12	Probes: Startup, Readiness and Liveness	Not started	
13	Requests, Limits and Quality of Service	Not started	
14	Deployments and ReplicaSets	Not started	
15	Rolling Updates and Rollback	Not started	
16	StatefulSets	Not started	
17	DaemonSets	Not started	
18	Jobs and CronJobs	Not started	
19	Workload Selection Guide	Not started	
20	Services and Stable Access	Not started	
21	Service Types and External Traffic	Not started	
22	DNS and Service Discovery	Not started	
23	Ingress	Not started	
24	Gateway API	Not started	
25	Kubernetes Network Model and CNI	Not started	
26	NetworkPolicy	Not started	
27	Volumes and Ephemeral Storage	Not started	
28	PersistentVolumes, Claims and StorageClasses	Not started	
29	CSI, Snapshots and Storage Operations	Not started	
30	Running Stateful Applications Safely	Not started	
31	Storage Troubleshooting	Not started	
32	ConfigMaps and Application Configuration	Not started	
33	Secrets and Workload Identity	Not started	
34	Scheduling: Node Selection and Affinity	Not started	
35	Taints, Tolerations and Dedicated Nodes	Not started	
36	Topology Spread and High Availability	Not started	
37	Autoscaling: HPA, VPA and Cluster Capacity	Not started	
38	Disruptions, Eviction and PodDisruptionBudgets	Not started	
39	ResourceQuota and LimitRange	Not started	
40	ServiceAccounts, Authentication and RBAC	Not started	
41	Security Contexts and Pod Hardening	Not started	
42	Pod Security Standards and Admission	Not started	
43	Admission Control and Policy Engines	Not started	
44	Helm Package Management	Not started	
45	Kustomize and Environment Overlays	Not started	

46	CRDs, Operators, Observability, GitOps and Cluster Operations	Not started	
47	Capstone 1: Production Application Platform	Not started	
48	Capstone 2: Secure Multi-Tenant Cluster Baseline	Not started	
49	Kubernetes Interview Mastery	Not started	
50	Final Practical Examination and Completion Review	Not started	

Final Certification Checklist

- ☐ All 50 units completed.
- ☐ All command laboratories reproduced in a disposable cluster.
- ☐ Every manifest stored in version control.
- ☐ Capstone 1 completed with rollout, rollback, backup and recovery evidence.
- ☐ Capstone 2 completed with tenancy, security and governance evidence.
- ☐ Interview scenarios practiced aloud.
- ☐ Final practical score is at least 80 percent.
- ☐ Weak areas were revised and revalidated.

Reference Baseline and Official Sources

Reference date: 30 July 2026. The course uses upstream Kubernetes documentation as the primary conceptual source. Key official references include the Kubernetes 1.36 release page, patch release schedule, cluster architecture and components, Pods, Kubernetes objects, security documentation, Pod Security Standards, Pod Security Admission and Gateway API documentation. Before production implementation, consult the exact target version and the documentation of every installed implementation.

- <https://kubernetes.io/releases/1.36/>
- <https://kubernetes.io/releases/patch-releases/>
- <https://kubernetes.io/docs/concepts/architecture/>
- <https://kubernetes.io/docs/concepts/overview/components/>
- <https://kubernetes.io/docs/concepts/overview/working-with-objects/>
- <https://kubernetes.io/docs/concepts/workloads/pods/>
- <https://kubernetes.io/docs/concepts/security/>
- <https://kubernetes.io/docs/concepts/security/pod-security-standards/>
- <https://kubernetes.io/docs/concepts/security/pod-security-admission/>
- <https://kubernetes.io/docs/concepts/services-networking/gateway/>

Personal Completion Declaration

I confirm that I completed the practical work and understand that real Kubernetes changes require authorization, peer review, version compatibility checks, least privilege, tested backups, observability and rollback planning.

Name: _____ Date: _____

Signature: _____